

A Practical Dynamic Programming Based Methodology for Aircraft Maintenance Check Scheduling Optimization

Qichen Deng*, Bruno F. Santos, Richard Curran

*Section of Air Transport and Operations
Delft University of Technology, The Netherlands*

Abstract

This paper presents a practical dynamic programming based methodology to optimize long-term maintenance check schedule for a fleet of heterogeneous aircraft. It is the first time that the long-term aircraft maintenance check schedule is optimized, integrating different check types in a single schedule solution. The proposed methodology aims at minimizing the wasted interval between checks. By achieving this goal, one is also reducing the number of checks over time, increasing aircraft availability and, therefore, reducing maintenance costs, while respecting safety regulations. The model formulation takes aircraft type, status, maintenance capacity, and other operational constraints into consideration. We also validate and demonstrate the proposed methodology using fleet maintenance data from a European airline. The outcomes show that, when compared with the current practice, the number of maintenance checks can be reduced by around 7% over a period of 4 years, while computation time is less than 15 min. This could result in saving worth \$1.1M-\$3.4M in maintenance costs for a fleet of about 40 aircraft and generating more than \$9.8M of revenue due to higher aircraft availability.

Keywords: Aircraft Maintenance, Dynamic Programming, Forward Induction, Scheduling Optimization

1. Introduction

Aircraft maintenance is the overhaul, repair, inspection or modification of an aircraft or aircraft systems, components and structures in an airworthy condition [1]. Regular maintenance prevents aircraft components and systems failures during operations. It takes place when an aircraft undergoes certain flight hours, flight cycles or calendar months. There are three major types of maintenance: A-check, C-check and D-check ¹. A typical A-check includes inspection of the interior or exterior of the airplane with selected areas opened (e.g., checking and servicing oil, filter replacement and lubrication) [2]; they are performed approximately every 2 to 3 months. The C-check requires thorough inspection of individual systems and components for serviceability and function; it is planned within an interval of 18 to 24 months. The D-check (a.k.a Structural

*Corresponding Author
Email address: q.deng@tudelft.nl (Qichen Deng)

¹B-checks are rarely mentioned in practice. The tasks that could be included in B-checks are commonly incorporated into successive A-checks

1
2
3
4
5
6
7
8
9
10 Check) uncovers the airframe, supporting structure and wings for inspection of most structurally significant
11 items; it is carried out every 6 to 10 years. Many airlines merge D-check into C-check and label it as a
12 heavy C-check. During a C-check or D-check, the aircraft has to be grounded for several weeks and removed
13 from the revenue schedule. This paper for the first time optimizes the long-term integrated A- and C-check
14 scheduling problem. We call this problem the aircraft maintenance check scheduling problem, or for short
15 the AMCS problem.

16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65

Scheduling the maintenance inspection for a large heterogeneous fleet is generally a demanding and complex problem. In practice, the aircraft maintenance schedules are usually prepared according to the experience of maintenance operators. The main problem associated with such planning approach is that it is time consuming and it can result in poor solutions. For a large fleet, the maintenance operators need to spend several days or weeks planning the A- and C-checks one after another according to individual aircraft inspection interval and maintenance resource of the airline. If conflict A- or C-checks occur, the maintenance operator needs to adjust the schedule, constantly moving checks to earlier or later time slots until a feasible schedule is found. Limited by the manual planning approach, the goal is usually to find a feasible maintenance schedule for a fleet instead of an optimal one [3]. As a result, the traditional manual maintenance planning approach inevitably decreases the aircraft utilization and leads to more A- and C-checks in the long term, increasing aircraft maintenance cost. Therefore, an optimized long-term maintenance schedule reduces the number of maintenance checks and increases aircraft availability, the saving derived from efficient maintenance planning can be very substantial.

Nowadays, airlines are laying increasing emphasis on improving their aircraft availability and planning their maintenance in a more efficient way. Aircraft maintenance represents one of the main direct operating costs and plays an important role on the balance sheet of an airline. According to [4], 11% of the total cost of an airline goes to aircraft maintenance. In 2014, this was equivalent to about \$295M per year per average airline [5]. The long-term economical and operational benefits of adopting a more efficient approach are clear; a typical C-check of A320 family may cost \$150k—\$350k [2], an A-check costs around \$10k—\$15k, while an additional day on operation may represent \$75k—\$120k of commercial revenue (depending on the utilization level of aircraft). However, the challenge is that A- and C-check scheduling problems are two correlated combinatorial problems. The decision of scheduling or not scheduling A-/C-check on an aircraft today impacts on the utilization of the aircraft onwards and, therefore, on the need to perform A-/C-checks in the future. This kind of problem is hard to solve and is often addressed by heuristics or algorithms [6].

In this paper we propose a dynamic programming (DP) based method to solve the AMCS problem. The main contribution of this work can be summarized in the following three aspects:

1) Methodology:

- A tractable DP-based model formulation is presented, suitable to solve large scale scheduling problems.

- A thrifty algorithm is used to infer future implications of an action taken in the current time stage.

2) Practicality

- The optimization takes the inspection interval of different check types and detailed operations into consideration.
- It takes less than 15 min to optimize the 4-year A- and C-check schedule for 40 aircraft, rather than days or weeks.

3) Application

- It is the first time that the long-term aircraft A- and C-check schedule is optimized by a single algorithm.
- The formulation is flexible and other maintenance events can be easily included in the proposed model, such as landing gear maintenance or cabin modification.

The outline of this paper is as follows: Section 2 reviews the literature about aircraft maintenance planning and solution techniques for scheduling problems. The aircraft maintenance requirements, constraints and AMCS problem formulation, for aircraft A- and C-check scheduling, are presented in Section 3. The DP based methodology for A- and C-check scheduling optimization is discussed in detail in Section 4. Section 5 describes the case study from a European airline. The last section summarizes the research with concluding remarks and gives an outlook to future work.

2. Literature Overview

Aircraft maintenance scheduling has been relying on the manual planning approach for many years. Since the introduction of commercialized wide-body aircraft in the early 1970s, the maintenance scheduling has become increasingly difficult due to the emphasis on efficiency and lack of an accurate and timely maintenance scheduling tool. It usually took several weeks for planning personnel to create a maintenance schedule [7]. Air Canada was aware of this issue and developed in the 1970's an aircraft maintenance operations simulation model (AMOS) to improve maintenance efficiency and reduce labor and material cost [7]. The AMOS tool formulated the aircraft maintenance scheduling as a discrete integer programming problem. According to the author, the problem constraints in AMOS included man-power, public holidays and summer time (when no maintenance was allowed), contractual maintenance duties for other airlines, reliability and required inspection intervals from the maintenance planning document (MPD). Several assumptions had been made in AMOS: each maintenance event ties up only one hangar/slot; the minimum time unit is one calendar day; aircraft is aged by daily flight hours; maintenance events can be postponed from the desired due date within a certain tolerance. Although AMOS works for both C-check and A-check scheduling with the objective of minimizing total unused flight hours between two successive C-checks, there was actually no optimization

technique involved. The method in AMOS is very similar to the manual planning approach, shifting conflict checks to earlier time slots until a feasible solution is found, except that a lower bound of utilization was implemented to prevent scheduling checks too often. Generally speaking, the algorithm in AMOS is a computerized manual planning approach. But considering the problem and constraints formulation, it is quite a significant achievement in maintenance scheduling.

Despite the limitations from AMOS, this is together with [8] and [9] the only available reference devoted to long-term aircraft maintenance scheduling. The long-term planning still has not been adequately studied. In fact, most research works about the aircraft maintenance topic focus on the short-term aircraft maintenance routing. That is, ensuring that each aircraft is assigned to a sequence of flight legs (a routing) that allows the aircraft to undergo daily checks, which are needed every two to four days [10]. The main reason is that aircraft A-/C-checks have intervals of several months/years and the benefits of an optimal schedule are only visible in the long term. Airlines usually have higher urgency to monitor and optimize short-term activities, such as aircraft routing and routine aircraft inspections, from which they can rapidly see tangible cost savings and profits.

Feo et al [11] is one of the first works to address the aircraft maintenance routing problem. It primarily focuses on the flight schedule design and incorporates the maintenance requirement as part of the constraints. A homogeneous fleet and fixed time intervals between maintenance checks were considered for simplicity. Only A-checks are considered in this work, since C-checks are spaced at relatively large time intervals. The planned flight schedule minimizes the total maintenance cost and also determines the maintenance base for the aircraft which start and end at the same city. The problem is formulated as a min-cost, multi-commodity flow network with integer constraints. Column generation is applied to obtain an optimized solution. Although the main purpose of the work [11] is to design a flight schedule, it is considered as a significant step in maintenance planning. Several authors have followed this path and continued the research on aircraft maintenance routing, such as [12], [13], [14] and [15]. For example, Moudani et al [12] proposed a hybrid dynamic programming (DP) which recursively searched for the best maintenance schedule, followed by a greedy algorithm to solve the sequential maintenance schedule problem. This approach was developed specifically as an on-line fleet operations management decision support system, focused on providing improved daily aircraft assignment solutions based on a given aircraft maintenance check schedule.

Sriram et al [16] is one rare example where the focus has been shifted from flight schedule design to maintenance scheduling. The authors proposed a mixed random search and depth first search heuristic to minimize the total costs of A- and B-checks and inappropriate aircraft assignments. Different from other research works which emphasize flight schedule design and consider aircraft maintenance as constraint, the flight schedule is given as input and, the goal is to determine when, where and what type of maintenance check an aircraft should undergo. Although the main focus is on maintenance scheduling, this research is still considered as an extension of [11], the C-check scheduling has not been considered since long-term flight schedules are unknown.

Instead of scheduling letter checks (A-, C- and structural/D-check), an alternative that reduces maintenance cost and generates profits in the short-term is aircraft maintenance task scheduling. This recent approach reverses the conventional top-down stereotyped planning. It schedules tasks individually, which gives flexibility to maintenance operators to execute the tasks at the most appropriate time [17][18]. The task-oriented planning concept and its application are illustrated in [19]. The case study claims that more than \$4M can be saved over 72 days compared with the rigid letter checks. However, among the works concerning task-oriented planning approach, there is little information about the influence of fleet size, maintenance capacity and algorithm computation time. Since an aircraft can have about 2000-3000 maintenance tasks, the practicality of applying a task-oriented approach on planning a long-term maintenance schedule for a large fleet remains questionable.

In general, the literature in long-term maintenance scheduling is limited, when planning aircraft maintenance checks, researchers often resort to the solution techniques from more general scheduling problems. A list of objective functions, models and optimization methods of scheduling problems are summarized by Duffuaa and Al-Sultan [20]. Since scheduling problems usually involve integer decision variables and linear constraints, one of the common approaches to such mixed-integer linear programming problems is dynamic programming (DP). DP was proposed in the 1950s by Bellman, referring specifically to nesting smaller decision problems inside larger decisions [21]. It divides a large and complicated problem into stages and states. The smaller sub-problems contained within each state are solved faster than the initial problem and the optimal solution can be retrieved by examining the solutions from all sub-problems. DP was initially applied on single-machine production [22][23][24] or single-machine maintenance scheduling problems [25]. For example, the work from Bomberger [22], which minimizes the total cost of producing different items from a single machine, is considered to be one of the first to motivate application of DP on scheduling problems, although it assumes that only one unit can be produced at a time and the demand rate of units is constant.

As the development of DP, the application has been gradually extended from scheduling of single machine to multiple machines. Most of the DP applications on multiple machines are related to power generation. One of the examples can be found in [26]. It presented a study to optimize the cost of multiple reservoir systems over a long period considering the uncertainties of water inflow and equipment outage. Even though piecewise linear functions are introduced in the solution process to estimate the future operation cost (this avoids recursively computing the actual future operating cost), the application is still limited to low-dimensional problems (a small number of reservoirs) [27]. When the number of reservoirs increases, the number of decisions, i.e., how much water should be kept in each reservoir, increases exponentially.

Several conclusions can be drawn from the review of literature. First of all, [7] is the only available reference for long-term A- and C-check scheduling although no optimization techniques is applied. Secondly, the long-term A- and C-check scheduling forms a typical large-scale combinatorial problem, but there is no standard approach or exact algorithm for such a problem type. Thirdly, aircraft A- and C-check scheduling on a fleet aggregate level is analogous to multiple unit scheduling, which can be treated with similar formulation

and solution techniques. Fourthly, DP is capable of dealing with small-scale mixed-integer/combinatorial problems, but the classic DP is not applicable to large-scale problems. In the rest of the paper, we will present a DP based methodology to tackle the long-term aircraft A- and C-check scheduling optimization.

3. Problem Formulation

In this section we formulate the AMCS problem, adopting the DP framework and the nomenclature presented in [Appendix A](#). We start with an introduction of maintenance inspection intervals (Subsection 3.1), followed by a list of assumptions (3.2). After that we explain the maintenance capacity and some common operational constraints (3.3). The formulation of the AMCS problem is then described, divided into decision space (3.4), definition of state (3.5), state transition (3.6), constraints formulation (3.7) and the objective function (3.8). The final subsection summarizes the optimization model formulation.

3.1. Maintenance Inspection Interval

In aviation industry, aircraft are aged by daily utilization with respect to 3 different usage parameters, calendar day (DY), flight hours (FH) and flight cycles (FC). One DY is a full 24 hours period; FH refers to the elapsed time between wheel lift off and touch down; and a FC is defined by a complete take-off and landing sequence. The inspection interval reflects to the maximum usage parameters allowed in operation. For example, the maintenance planning document (MDP) of the AIRBUS A320 family [28] defines that an C-check interval corresponds to 730 DY, 7500 FH or 5000 FC; and 120 DY, 750 FH or 750 FC for the A-check.

After an A-/C-check the corresponding three usage parameters are set to 0 and a maintenance cycle is concluded. These maintenance cycles are associated with labels, referring to different task packages (i.e., A1, A2, A3,... for the A-check and C1, C2,... for the C-check). The A-check program is commonly divided into 4 cycles, in which A1 has similar task packages as that of A5, while A2 has similar task packages as that of A6, and so forth. The C-check program has 12 cycles and consists of continuous C-checks, whereby every three check (i.e., C3, C6, C9, ...) is a heavier check incorporating tasks from the D-check.

The aircraft MPD also includes an inspection interval tolerance. This tolerance allows operators to fit the maintenance schedule around maintenance capacity and operations constraints, as well as operations demand. However, in the case that tolerance is used in one maintenance cycle, the amount of DY, FH and FC used from the tolerance need to be deducted from the maximum usage parameter values for the next cycle. This guarantees that the maximum usage parameters are verified in the long term. The inspection interval tolerance should not be included as a planning option but it is commonly used in practice to accommodate deviations from the initial schedule.

Although having different usage parameters, the A-check and C-check scheduling problems are dependent on each other due to two reasons: the first is that when an aircraft is performing a C-check (or A-check), it will be grounded and the A-check (or C-check) usage parameters are not altered (i.e., the daily utilization of these parameters is equal to 0). The second reason is that, depending on the usage parameters for the

1
2
3
4 A-check, it could be beneficial for the airline to merge the A-check within a C-check. This has the advantage
5 of performing the A-checks without necessarily increasing the C-check duration and without using an A-check
6 slot. On the other hand, to anticipate an A-check, merging an A-check within a C-check will increase the
7
8
9 number of A-checks in the long-term.

10 11 3.2. Assumptions 12

13 [7] defined a list of *major conditions* for maintenance event scheduling, based aircraft maintenance practice.
14 In this paper, we adopt the first five of these assumptions (A.1–A.5) and add three more (A.6–A.8) necessary
15
16
17 to define our approach.
18

19 **A.1** Minimum time unit of the aircraft maintenance schedule is 1 DY.
20

21 **A.2** Aircraft ages by DY, daily FH and FC. The daily utilization, as well as the commercial peak seasons,
22
23 can be estimated per aircraft according to historical data.
24

25 **A.3** Each A-/C-check ties up only one hangar (slot) for its total duration.
26

27 **A.4** C-check has higher priority than A-check.
28

29 **A.5** A-check can be merged in C-check, which will not affect the C-check duration or existing A-check slots.
30

31 **A.6** Aircraft with higher A-/C-utilization have higher A-/C-check priority.
32

33 **A.7** The duration of an A-/C-check per check label can be estimated according to historical data or can be
34
35 specified by airline.
36

37 **A.8** There is flexibility in the aircraft routing to accommodate the A-/C-check.
38
39

40 The last assumption is based on the fact that the AMCS problem is a strategic problem. The aircraft
41
42 routes are still not known, since they are only defined a couple of weeks before operations.
43

44 3.3. Maintenance Capacity and Operational Constraints 45

46 In the AMCS problem, the A-/C-check capacity can either be expressed as person-hour available during
47
48 a working day or, equivalently, as the maximum parallel A-/C-check allowed per working day (defined as
49
50 ‘time slots’ or just ‘slots’). The capacity for each check type is not always constant over time. Airlines
51 and maintenance, repair and overhaul (MRO) service providers usually have operational constraints that
52 influence the maintenance capacity per day. For instance, during commercial peak season (e.g., the period of
53 New Year, Easter, summer and Christmas) it is desirable for the airline to operate with the maximum fleet.
54
55 Performing C-checks will lead to high commercial revenue loss and it is also common to have reduced or no
56
57 checks during weekends and public holidays due to higher labor costs. A final example is the case where
58
59 some maintenance capacity is pre-allocated to third-party aircraft and therefore cannot be considered in the
60
61 airline maintenance schedule.
62
63
64
65

In this paper, we define the capacity $M_{h,t}^k$ for hangar h :

$$M_{h,t}^k = \begin{cases} 1 & \text{if hangar } h \text{ is available on day } t \text{ for type } k \text{ slots} \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

This parameter has to be defined per day per hangar for the entire time horizon, reflecting capacity variations between peak season and off-peak season and between weekends and normal working days, according to the airline policy. The capacity $M_{h,t}^k$ for hangar h can also be set equal to zero if hangar h is reserved for a specific maintenance event, such as performing a landing gear change on an aircraft of the fleet or for a type k check of an third-party aircraft.

3.4. Decision Space

An action x_t of day t is to perform A-checks or C-checks, or do nothing:

$$x_t = \left\{ \left\{ \chi_{i,t}^k \right\}_{i=1}^N \right\}_{k \in \{A, C\}} \quad (2)$$

where, each $\chi_{i,t}^k$ is a binary decision variables in which:

$$\chi_{i,t}^k = \begin{cases} 1 & \text{a type } k \text{ check for aircraft } i \text{ is planned to start at time } t \\ 0 & \text{otherwise} \end{cases} \quad (3)$$

3.5. Definition of State

A state vector s_t is defined by the set of attributes that influence our decisions and the available maintenance slots of each check type:

$$s_t = \left\{ \left\{ a_{i,t}^A \right\}_{i=1}^N, \left\{ a_{i,t}^C \right\}_{i=1}^N \right\} \quad (4)$$

where, each attribute set $a_{i,t}^k$ contains the information of aircraft i on day t , with respect to check type k :

$$a_{i,t}^k = \underbrace{\{M_t^k, z_{i,t}^k, \delta_{i,t}^k, \eta_{i,t}^k\}}_{\text{Type 1 } (a_{i,t}^{(1),k})} \underbrace{\{DY_{i,t}^k, FH_{i,t}^k, FC_{i,t}^k, \epsilon_{i,t}^{k-DY}, \epsilon_{i,t}^{k-FH}, \epsilon_{i,t}^{k-FC}, \theta_{i,t}^k, y_{i,t}^k\}}_{\text{Type 2 } (a_{i,t}^{(2),k})} \underbrace{\{L_i^k(y_{i,t}^k), fh_{i,t}, fc_{i,t}\}}_{\text{Type 3 } (a_{i,t}^{(3),k})} \quad (5)$$

These attributes are described in [Appendix A](#) and discussed in the next subsection. They can be divided into three types, as showed in Table 1.

3.6. State Transition

The transition between states in subsequent time steps depends on the actions taken. This can be described by a *state transition function* in which the state s_{t+1} is defined as a function of the initial state s_t and the action x_t chosen in state s_t :

$$\begin{cases} x_t = \mathcal{X}^\pi(s_t) \\ s_{t+1} = \mathcal{S}^\mathcal{X}(s_t, x_t) \end{cases} \quad \text{for } t = t_0, t_0 + 1, \dots, T \quad (6)$$

Table 1: Different types of attribute within a state s_t .

Type 1 $a_{i,t}^{(1),k}$	Attributes at time t that impact the action x_t and are modified only when a check starts or ends, or when an aircraft is grounded
Type 2 $a_{i,t}^{(2),k}$	Attributes at time t that are updated every time based on their value at time $t - 1$
Type 3 $a_{i,t}^{(3),k}$	Attributes at time t that depend on exogenous information

where $\mathcal{X}^\pi(s_t)$ generates actions based on s_t according to A- and C-check hangar capacities at day t , M_t^A and M_t^C . The state transition function $\mathcal{S}^\mathcal{X}(s_t, x_t)$ describes how the state vector is updated and expresses the fact that an action taken at time t influences the future maintenance activities and capacities. A history of such process, including the sequence of actions and evolution of state, can be represented as:

$$(s_{t_0}, x_{t_0}, s_{t_0+1}, x_{t_0+1}, s_{t_0+2}, \dots, s_{t-1}, x_{t-1}, s_t, \dots, s_T, x_T, s_{T+1}) \quad (7)$$

The main purpose of state transition is to renew the attributes over the time horizon. The attributes are updated in two phases: pre-decision (Phase 1) and post-decision (Phase 2). The goal of the pre-decision phase is to update the hangar capacity and aircraft availability for time t , before any decision is made. This provides the information about how many hangars can be used to perform A-/C-checks and which aircraft is available for operation. In Phase 1, only Type 1 attribute ($a_{i,t}^{(1),k}$) is updated and the resulting attributes ($\tilde{a}_{i,t}^{(1),k}$) from the pre-decision update is defined as pre-decision attributes. On the other hand, the goal of the post-decision phase is to update aircraft usage parameters, according to the action x_t that made on day t . All 3 type attributes will be updated in Phase 2 and we call the subsequent attributes from Phase 2 update post-decision attributes. Since the attributes of a state are divided into three types (Table 1), the transition of each type of attributes is presented separately in the following sub-sections.

3.6.1. Update of Type 1 Attributes

Phase 1, which we called the pre-decision phase (Figure 1), only updates the Type 1 attributes $a_{i,t}^{(1),k}$. In this phase, we check if at time t is the end day for an ongoing aircraft check. The new vector within the pre-decision attributes of type 1 is $\tilde{a}_{i,t}^{(1),k} = \{\tilde{M}_t^k, \tilde{z}_{i,t}^k, \tilde{\delta}_{i,t}^k, \tilde{\eta}_{i,t}^k\}$. The pre-decision update is triggered by verifying if the end date ($z_{i,t}^k$) of an ongoing check is equal to $t - 1$ (i.e., if $z_{i,t}^k = t - 1$), for any aircraft in the fleet:

$$\tilde{z}_{i,t}^k = \begin{cases} 0 & \text{if } z_{i,t}^k = t - 1 \\ z_{i,t}^k & \text{otherwise} \end{cases} \quad (8)$$

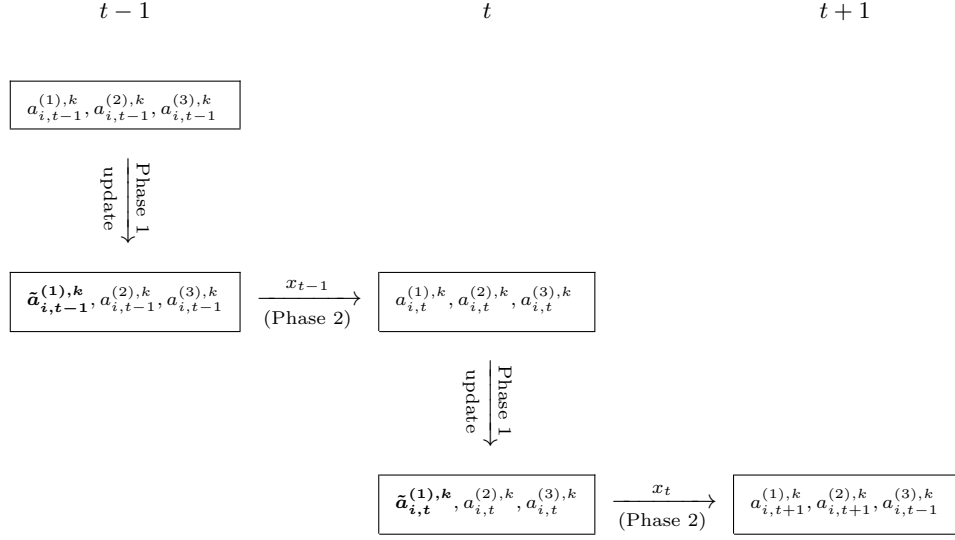


Figure 1: A two-phase attribute update mechanism: Phase 1 (Ph-1) updates the set of pre-decision Type 1 attribute $a_{i,t}^{(1),k}$ to $\tilde{a}_{i,t}^{(1),k}$ before making any action; after an action x_t is made, Phase 2 (Ph-2) updates $\tilde{a}_{i,t}^{(1),k}$, $a_{i,t}^{(2),k}$ and $a_{i,t}^{(3),k}$ to $a_{i,t+1}^{(1),k}$, $a_{i,t+1}^{(2),k}$ and $a_{i,t+1}^{(3),k}$.

At the same time, we update $\delta_{i,t}^k$ to $\tilde{\delta}_{i,t}^k$:

$$\tilde{\delta}_{i,t}^k = \begin{cases} 0 & \text{if } z_{i,t}^k = t-1 \\ 1 & \text{otherwise} \end{cases} \quad (9)$$

If the end date of a type k check for an aircraft i is larger than the current calendar day t , it means that there is an aircraft check occurring. And the hangar capacity needs to be updated for time t accordingly:

$$\tilde{M}_t^k = \sum_h M_{h,t}^k - \sum_{i=1}^N \tilde{\delta}_{i,t}^k \quad (10)$$

where $M_{h,t}^k$ is the maintenance capacity per hangar h at time t . The value of $\tilde{\eta}_{i,t}^k$ is initialized using $\eta_{i,t}^k$, namely, $\tilde{\eta}_{i,t}^k = \eta_{i,t}^k$.

In Phase 2, or the post-decision phase (see Figure 1), the action x_t is taken into account to update Type 1 attributes. For all aircraft that start type k check on day t ($\chi_{i,t}^k = 1$), the values of $\delta_{i,t}^k$ and $z_{i,t}^k$ need to be updated. The $z_{i,t}^k$ is updated according to:

$$z_{i,t+1}^k = \begin{cases} t + L_i^k(y_{i,t}^k) & \text{if } \chi_{i,t}^k = 1 \\ \tilde{z}_{i,t}^k & \text{otherwise} \end{cases} \quad (11)$$

where $L_i^k(y_{i,t}^k)$ is the elapse time for maintenance type k , with label $y_{i,t}^k$. Following this update, the values of $\delta_{i,t}^k$ can also be renewed:

$$\delta_{i,t+1}^k = \begin{cases} 0 & \text{if } \chi_{i,t}^k = 1 \\ \tilde{\delta}_{i,t}^k & \text{otherwise} \end{cases} \quad (12)$$

Still in the post-decision stage, in some special cases, aircraft reach their inspection intervals and no maintenance check capacity is available. In these undesirable situations, the aircraft needs to be grounded and put out of operations, waiting for the next maintenance opportunity. This happens if the usages parameters for time $t + 1$ of any aircraft is larger than the respective inspection interval. We first compute the expected usage parameters of $t + 1$ as follows:

$$\Delta DY_{i,t+1}^k = \underbrace{\left(DY_{i,t}^k + 1 \right)}_{DY_{i,t+1}^k} - \underbrace{\left[I_{k-DY}^i + (1 - \theta_{i,t}^k) e_{k-DY}^i - \epsilon_{i,t}^{k-DY} \right]}_{\text{Actual DY Interval of Type } k \text{ Check}} \quad (13)$$

$$\Delta FH_{i,t+1}^k = \underbrace{\left(FH_{i,t}^k + fh_{i,t} \right)}_{FH_{i,t+1}^k} - \underbrace{\left[I_{k-FH}^i + (1 - \theta_{i,t}^k) e_{k-FH}^i - \epsilon_{i,t}^{k-FH} \right]}_{\text{Actual FH Interval of Type } k \text{ Check}} \quad (14)$$

$$\Delta FC_{i,t+1}^k = \underbrace{\left(FC_{i,t}^k + fc_{i,t} \right)}_{FC_{i,t+1}^k} - \underbrace{\left[I_{k-FC}^i + (1 - \theta_{i,t}^k) e_{k-FC}^i - \epsilon_{i,t}^{k-FC} \right]}_{\text{Actual FC Interval of Type } k \text{ Check}} \quad (15)$$

where, $DY_{i,t}^k$, $FH_{i,t}^k$ and $FC_{i,t}^k$ are the cumulative DY, FH and FC since previous type k check till day t ; I_{k-DY}^i , I_{k-FH}^i and I_{k-FC}^i refer to the standard interval of type k check; $(1 - \theta_{i,t}^k) e_{k-DY}^i$, $(1 - \theta_{i,t}^k) e_{k-FH}^i$ and $(1 - \theta_{i,t}^k) e_{k-FC}^i$ represent the respective tolerance that can be added to the standard interval; and $\epsilon_{i,t}^{k-DY}$, $\epsilon_{i,t}^{k-FH}$ and $\epsilon_{i,t}^{k-FC}$ represent the amount of DY, FH and FC tolerance used in previous type k check (and that needs to be deducted from the coming maintenance check according to the aircraft MPD).

The aircraft is then grounded if any of these previous delta values is greater than 0 and no maintenance check is being performed on the aircraft:

$$\eta_{i,t+1}^k = \begin{cases} 1 & \chi_{i,t}^k = 0, \max \left\{ \Delta DY_{i,t+1}^k, \Delta FH_{i,t+1}^k, \Delta FC_{i,t+1}^k \right\} > 0 \\ \tilde{\eta}_{i,t}^k & \text{otherwise} \end{cases} \quad (16)$$

3.6.2. Update of Type 2 Attributes

Once the action of day t is known, the update of Type 2 attributes is trivial. The aircraft usage parameters are updated according to the following equations:

$$DY_{i,t+1}^k = (1 - \delta_{i,t}^k) \left(DY_{i,t}^k + 1 \right) \quad (17)$$

$$FH_{i,t+1}^k = (1 - \delta_{i,t}^k) \left[FH_{i,t}^k + \left(1 - \delta_{i,t}^{\{A,C\} \setminus k} \right) fh_{i,t} \right] \quad (18)$$

$$FC_{i,t+1}^k = (1 - \delta_{i,t}^k) \left[FC_{i,t}^k + \left(1 - \delta_{i,t}^{\{A,C\} \setminus k} \right) fc_{i,t} \right] \quad (19)$$

where $\delta_{i,t}^{\{A,C\} \setminus k}$ means if $k = A$ then $\delta_{i,t}^{\{A,C\} \setminus k} = \delta_{i,t}^C$ and vice versa. The parameters are reset to 0 if a maintenance check of type k was scheduled in the previous time step (i.e., $\delta_{i,t}^k = 1$). Otherwise, the parameters are either increased by the average daily aging of the aircraft or kept constant, if a maintenance of the type other than k is scheduled (i.e., $\delta_{i,t}^{\{A,C\} \setminus k} = 1$).

The update of Type 2 attributes also includes renewing the tolerance usage variables for each maintenance check type k :

$$\epsilon_{i,t+1}^{k-DY} = \begin{cases} \max\{0, DY_{i,t}^k - I_{k-DY}^i\} & \text{if } \chi_{i,t}^k = 1 \\ \epsilon_{i,t}^{k-DY} & \text{otherwise} \end{cases} \quad (20)$$

$$\epsilon_{i,t+1}^{k-FH} = \begin{cases} \max\{0, FH_{i,t}^k - I_{k-FH}^i\} & \text{if } \chi_{i,t}^k = 1 \\ \epsilon_{i,t}^{k-FH} & \text{otherwise} \end{cases} \quad (21)$$

$$\epsilon_{i,t+1}^{k-FC} = \begin{cases} \max\{0, FC_{i,t}^k - I_{k-FC}^i\} & \text{if } \chi_{i,t}^k = 1 \\ \epsilon_{i,t}^{k-FC} & \text{otherwise.} \end{cases} \quad (22)$$

(20)–(22) indicate that the status of tolerance usage of a type k check is the same as the day before if there is no type k check allocated on day t . On the contrary, if a type k check is scheduled before all usage parameters reach maximum, then no tolerance is used and $\epsilon_{i,t+1}^{k-DY}/\epsilon_{i,t+1}^{k-FH}/\epsilon_{i,t+1}^{k-FC}$ are set to 0. If an aircraft has to operate over the limit of a type k check, the corresponding $\epsilon_{i,t+1}^{k-DY}/\epsilon_{i,t+1}^{k-FH}/\epsilon_{i,t+1}^{k-FC}$ are updated according to the difference between the cumulative DY/FH/FC and type k check interval. As a result, the tolerance usage indicators will be renewed:

$$\theta_{i,t+1}^k = \begin{cases} 1 & \text{if } \max\{\epsilon_{i,t+1}^{k-DY}, \epsilon_{i,t+1}^{k-FH}, \epsilon_{i,t+1}^{k-FC}\} > 0 \\ 0 & \text{otherwise.} \end{cases} \quad (23)$$

After an action is evaluated, the maintenance labels for both type k checks are updated consequently. The maintenance labels of an aircraft i are updated to the next label using the following equation:

$$y_{i,t+1}^k = \begin{cases} y_{i,t}^k + 1 & \text{if } \chi_{i,t}^k = 1 \\ y_{i,t}^k & \text{otherwise} \end{cases} \quad (24)$$

3.6.3. Update of Type 3 Attributes

The Type 3 attributes are exogenous variables that are updated according to lookup tables, or provided by an airline, or estimated according to historical data of airline. They refer to:

- $L_i^k(y_{i,t+1}^k)$ is the elapsed time specified by airline.
- $fh_{i,t+1}^k$ is estimated according to historical aircraft FH.
- $fc_{i,t+1}^k$ is estimated according to historical aircraft FC.

3.7. Constraints Formulation

There are two types of constraints in the A- and C-check scheduling optimization: A-/C-check intervals and A-/C-check operational constraints. The A-/C-checks are usually scheduled before the corresponding usage parameters reach maximum. That not being possible, in practice the airline can make use of the

interval tolerance. The extra DY/FH/FC used from tolerance must be compensated in the next type k check, as mentioned in Subsection 3.6.1. This can be described as follows, for each check k , aircraft i , and time t :

$$\text{DY}_{i,t+1}^k \leq I_{k\text{-DY}}^i + (1 - \theta_{i,t}^k) e_{k\text{-DY}}^i - \epsilon_{i,t}^{k\text{-DY}} \quad (25)$$

$$\text{FH}_{i,t+1}^k \leq I_{k\text{-FH}}^i + (1 - \theta_{i,t}^k) e_{k\text{-FH}}^i - \epsilon_{i,t}^{k\text{-FH}} \quad (26)$$

$$\text{FC}_{i,t+1}^k \leq I_{k\text{-FC}}^i + (1 - \theta_{i,t}^k) e_{k\text{-FC}}^i - \epsilon_{i,t}^{k\text{-FC}} \quad (27)$$

where the first term of the right-hand side of each inequality refers to the standard check interval, the second terms adds the tolerance interval, and the last term subtracts the tolerance used in the previous check of the same type.

Before instigating any action, we need to verify whether or not there are sufficient slots for a type k check in a hangar during the entire maintenance elapse time $L_i^k(y_{i,t}^k)$, for all aircraft and hangars available:

$$\chi_{i,t}^k \leq \frac{\sum_{\tau=t}^{t+L_i^k(y_{i,t}^k)} M_{h,\tau}^k}{L_i^k(y_{i,t}^k)}, \quad k \in \{A, C\}, \quad t \in [t_0, T] \quad (28)$$

The operational constraints are required to guarantee that the number of A-/C-checks performed in parallel per day do not exceed the hangar capacity, namely:

$$\sum_{i=1}^N \delta_{i,t}^k \leq \sum_h M_{h,t}^k, \quad k \in \{A, C\}, \quad t \in [t_0, T] \quad (29)$$

Some airlines require a minimum number of days (d_k) between the start dates of two type k checks preparing the maintenance resources, such as for tools, manpower, aircraft spare parts and to avoid parallel peaks of workload at the hangar, meaning that:

- If $d_k > 0$, there can be at most 1 aircraft starts a type k check at time t .
- If $d_k > 0$ and there is a type k check starting at t , no type k check is allowed to start in $[t, t + d_k)$

The requirement of start date can be translated in the following equations for all time t :

$$\sum_{i=1}^N \chi_{i,t}^k \leq \begin{cases} 1 & \text{if } d_k > 0 \text{ and } \sum_{i=1}^N \chi_{i,\tau}^k = 0, \forall \tau \in [t - d_k, t) \\ M_t^k & \text{otherwise} \end{cases} \quad (30)$$

3.8. Objective Function

When scheduling aircraft maintenance activities, the most common objectives are minimization of costs [12][16] or minimization of the unused flight hours (FH) [7][14]. In this work we consider the second objective. The cost minimization objective was not considered for three main reasons:

- The available maintenance cost data is unreliable and hard to associate to a specific maintenance check;
- Maintenance checks are mandatory and the total maintenance costs of an airline can only be reduced if the number of aircraft checks over time is also reduced;
- One day of an aircraft out of operations is more costly than the daily cost of a maintenance check.

Therefore, minimization of the unused FH, and consequently, in the long term, the reduction of the number of aircraft checks and days on the ground, is considered to be the best objective for the AMCS problem. For an aircraft i , the value of unused FH in a day t is equal to the summation of the FH loss due to an A- or a C-check scheduled for that day:

$$\chi_{i,t}^A (I_{A-FH}^i - FH_{i,t-1}^A) + \chi_{i,t}^C (I_{C-FH}^i - FH_{i,t-1}^C) \quad (31)$$

The *contribution function* of FH loss on day t is calculated by:

$$C_t(s_t, x_t) = \sum_{k \in \{A, C\}} \sum_{i=1}^N \left[\chi_{i,t}^k (I_{k-FH}^i - FH_{i,t}^k) + (1 - \chi_{i,t}^k) P_a \theta_{i,t}^k + P_d \eta_{i,t}^k \right] \quad (32)$$

where the first term on the right hand side reflects the unused FH of aircraft i , the second term is a penalty for aircraft i using an interval tolerance, and the third term is a penalty for having an aircraft on the ground without doing maintenance.

The penalty P_a is introduced due to the fact that the use of tolerance needs to be communicated and approved by the local civil aviation authorities. Therefore, tolerance should not be considered at a scheduling stage or, if inevitable, it should be used as little as possible. The second penalty is introduced to reflect the cost of having an aircraft on the ground waiting for a maintenance slot. This results in very high costs and it should always be avoided, unless it proves to be unfeasible otherwise. For that reason, the value of P_d should always be of a very large magnitude. Our objective is then to minimize the sum of the total contributions for all states visited during the time horizon, discounted by a factor γ . That is, we search for the optimal AMCS policy (π) that minimizes the contribution of our scheduling decisions over the time horizon $T - t_0$:

$$\min_{\pi} \mathbb{E} \left\{ \sum_{t=t_0}^T \gamma^{t-t_0} C_t(s_t, \mathcal{X}^{\pi}(s_t)) \right\} \quad (33)$$

where $\mathcal{X}^{\pi}(s_t)$ is the optimal scheduling policy function.

3.9. Optimization Model

After the introduction of state transition, constraints and objective function, the optimization problem can be described by the following:

$$\min_{\pi} \mathbb{E} \left\{ \sum_{t=t_0}^T \gamma^{t-t_0} C_t(s_t, \mathcal{X}^{\pi}(s_t)) \right\} \quad (34)$$

subject to:

Constraints (11) – (30)

The optimal scheduling policy over the time horizon T can be found by recursively computing the Bellman's equation:

$$V_t(s_t) = \min_{x_t} \left\{ C_t(s_t, x_t) + \gamma \sum_{s_{t+1}} p(s_{t+1}|s_t, x_t) V_{t+1}(s_{t+1}) \right\} \quad (35)$$

where $s_{t+1} = \mathcal{S}^X(s_t, x_t) = \mathcal{S}^X(s_t, \mathcal{X}^\pi(s_t))$ and $p(s_{t+1}|s_t)$ is the probability of transitioning from state s_t to state s_{t+1} . The Bellman's equation expresses the value of being at each state S_t .

4. Methodology

The AMCS problem has a structure that follows the Markov Decision Process (MDP). Like any other MDP, the AMCS problem can be solved using dynamic programming (DP). The problem can be divided into stages, each stage referring to one calendar day (indexed by t). For each stage, all feasible actions x_t from a state s_t need to be evaluated and the optimal one x_t^* can be eventually identified.

The value associated with each action can be computed using Bellman's optimality equations (35). However, solving these for all possible actions is not trivial due to three challenges: the size of the multi-dimensional action vector x_t ; the size of the multi-dimensional state vector s_t ; and the very large outcome space. These challenges are well known as the ‘‘curse of dimensionality’’ [29]. It is easy to understand these challenges if we analyze our AMCS problem, as the state vector s_t is a tuple that contains the states of N aircraft, and each aircraft has 28 attributes with respect to the A- and C-check. If one wants to use discretization for each attribute, e.g. to l levels, the total number of levels to access will be $l^{28} \times N$, just for a single stage. This requires a large amount of computer memory and also makes it difficult to trace decisions backwards. In terms of actions, for each time stage t and capacities M_t^A (A-check) and M_t^C (C-check), we would have:

$$\sum_{m_c=0}^{M_t^C} \sum_{m_a=0}^{M_t^A} \left[\frac{N!}{(N-m_a)! \times m_a!} \times \frac{N!}{(N-m_c)! \times m_c!} \right] \quad (36)$$

possible actions and, if no optimal final state is given, there will be:

$$\prod_{t=t_0}^T \sum_{m_c=0}^{M_t^C} \sum_{m_a=0}^{M_t^A} \left[\frac{N!}{(N-m_a)! \times m_a!} \times \frac{N!}{(N-m_c)! \times m_c!} \right] \quad (37)$$

possible outcomes for the last stage. This means that even for a small case with 10 aircraft and one daily slot available for each check type, we would have 121 possible actions on the first day and more than 1.7 million possible sequences of actions just after three days.

In classic dynamic programming, computing $V_{t+1}(s_{t+1})$ requires $V_{t+2}(s_{t+2})$, and in order to obtain $V_{t+2}(s_{t+2})$, the $V_{t+3}(s_{t+3})$ has to be computed and so forth, until t reaching the final stage T . The aforementioned solution process is called backward induction that, if the final state is not known, easily becomes intractable even for the small example of 10 aircraft. This forces us to treat the AMCS problem in a different way, adopting a *forward induction* approach which moves from the initial planning stage towards the future.

In this section we propose a forward induction DP based methodology to solve the AMCS problem. We begin with a brief introduction to the forward induction concept in Subsection 4.1. After that we describe a priority solution that is proposed in dealing with the multi-dimensional action vector (Subsection 4.2). Subsection 4.3 presents a *Thrifty Algorithm* for A- and C-check scheduling, which estimates the implications of an action at the current stage on the remaining planning horizon. Subsection 4.4 presents the discretization and aggregation approach adapted to implement the algorithm. The last subsection (Subsection 4.5) includes an algorithm complexity analysis.

4.1. Forward Induction

Since the final state of our AMCS problem is not known, we propose the use of a forward induction approach. Forward induction is the process of reasoning forward in time, determining a sequence of optimal actions from an initial state till the end of the time horizon. This comes from the observation that the shortest path from an initial node s_{t_0} to an end node s_{T+1} is equal to the shortest path from the end node to the initial node [30]. That is, determining the optimal solution for the forward shortest path problem is the same as determining the optimal solution for the backward shortest path problem, as computed by the backward induction approach. The idea from the forward induction approach is to move forward in time, continuously computing the shortest path between the initial node s_{t_0} and the current node being tested s_t . This process is repeated until one has determined the best action for every stage in the time horizon.

Although the forward induction approach would solve the problem of not knowing the final state of our problem, due to the large number of intermediate states between the initial stage and the final stage of planning horizon, this approach is still inefficient for AMCS in terms of computation time and storage. For this reason, we incorporate forward induction with three additional components:

- An A- and C-check priorities definition solution
- A Thrifty Algorithm for A- and C-check scheduling
- A discretization and state aggregation strategy.

These components are labeled as blocks **A**, **B** and **C** respectively, in the work flow diagram of forward induction (Figure 2). The first component is used to deal with the multi-dimensional action vector; the second component is used to estimate the consequences of an action on future time steps; the third components is designed to reduce the outcome space to a manageable size. These three components are explained in the following subsections.

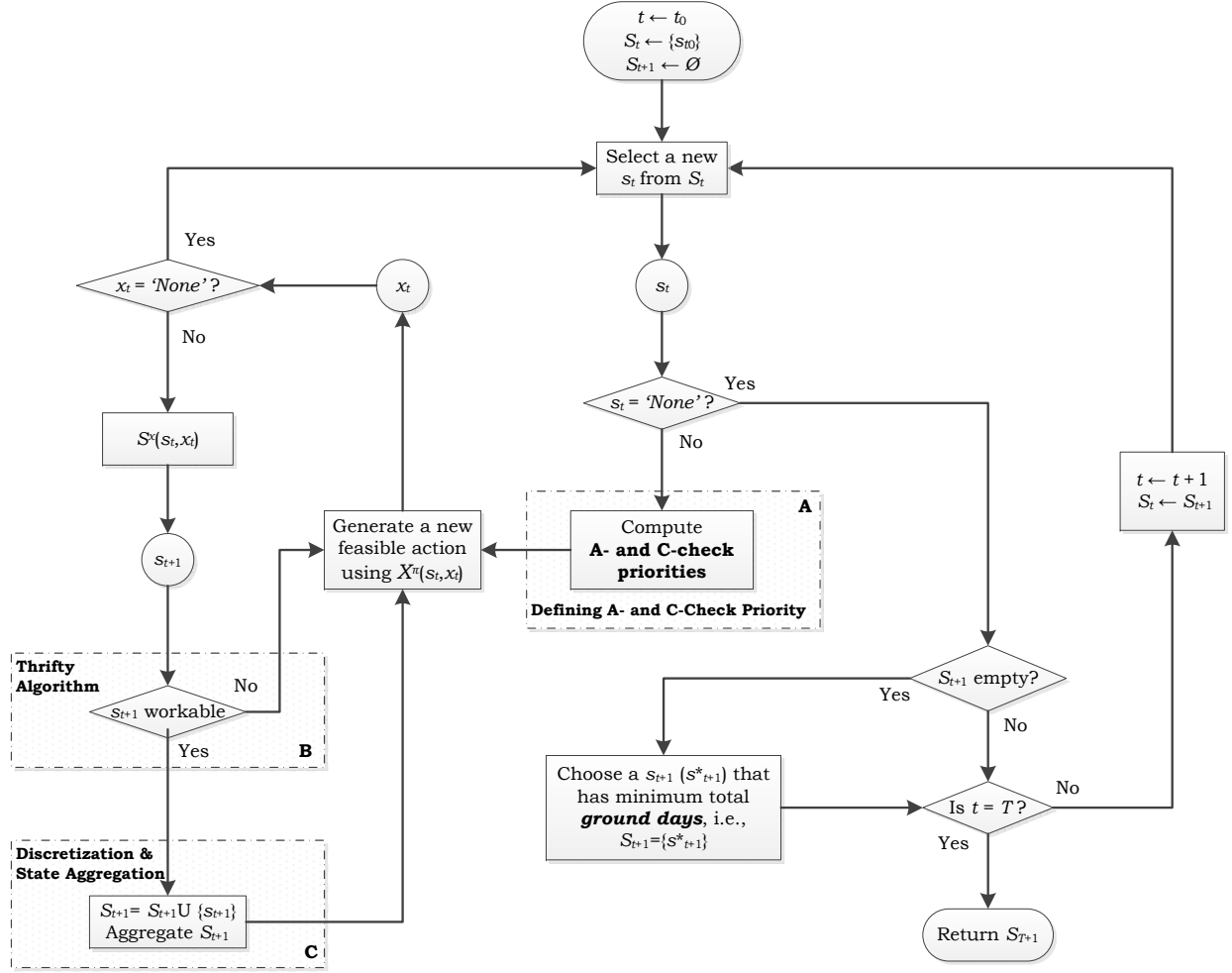


Figure 2: Work flow of the proposed DP based methodology. Three main components are incorporated, labeled as blocks **A**, **B** and **C**. In block **A**, we define the *A- and C-check priority* for each aircraft in order to reduce the number of actions. In block **B** we use a *Thrifty Algorithm* to infer future implications of an action and in block **C** we use *discretization and state aggregation* to overcome the challenge of multi-dimensional state vectors. The term *ground days* refers to the days when an aircraft is grounded and waiting for a A-/C-check slot. After the forward induction, a final state S_{t+1} is returned and a sequence of actions $x_{t_0}, x_{t_0+1}, \dots, x_T$ is generated.

4.2. Defining A-/C-Check Priority

Given a state s_t , we generate an action based on s_t :

$$x_t = \mathcal{X}^\pi(s_t) \quad (38)$$

Note that x_t can be performing m_C C-check and m_A A-check (m_C and m_A depend on corresponding hangar capacity), meaning that the action affects the status of multiple aircraft. This leads to a combination of aircraft selection and therefore, $\frac{(N!)^2}{m_C!(N-m_C)!m_A!(N-m_A)!}$ outcome states.

We observe that the number of states can quickly explode due to such a multi-dimensional action vector (being an action on multiple items). One common solution to this challenge is to assign priorities to each aircraft and in our case, we define A- and C-check priorities (**Block A** in Figure 2) according to the aircraft *remaining utilization*. Since there are 3 usage parameters for each check type in the AMCS problem, this gives 3 different remaining utilization with respect to DY, FH and FC, while a type k check should be scheduled before any of the remaining utilization goes to 0.

This way, we define the aircraft *remaining utilization* by the fewest days to the next A-/C-check:

$$R_{i,t}^k = \min \{ R_{i,t}^{k-DY}, R_{i,t}^{k-FH}, R_{i,t}^{k-FC} \} \quad (39)$$

The $R_{i,t}^{k-DY}$, $R_{i,t}^{k-FH}$ and $R_{i,t}^{k-FC}$ refer to the remaining operation days with respect to each usage parameter and associated interval specified by the MPD:

$$R_{i,t}^{k-DY} = \max_{r \in \mathbb{N}} \left\{ r \leq I_{k-DY}^i - \epsilon_{i,t}^{k-DY} - DY_{i,t}^k \right\} \quad (40)$$

$$R_{i,t}^{k-FH} = \max_{r \in \mathbb{N}} \left\{ \sum_{\tau=t}^{t+r} fh_{i,\tau} \leq I_{k-FH}^i - \epsilon_{i,t}^{k-FH} - FH_{i,t}^k \right\} \quad (41)$$

$$R_{i,t}^{k-FC} = \max_{r \in \mathbb{N}} \left\{ \sum_{\tau=t}^{t+r} fc_{i,\tau} \leq I_{k-FC}^i - \epsilon_{i,t}^{k-FC} - FC_{i,t}^k \right\} \quad (42)$$

where $\mathbb{N}$ is the set of natural numbers for $k \in \{A, C\}$. At any given time t , the remaining utilizations are sorted in ascending order:

$$\tilde{R}_{1,t}^k, \tilde{R}_{2,t}^k, \tilde{R}_{3,t}^k, \dots, \tilde{R}_{N,t}^k \quad \tilde{R}_{i,t}^k \leq \tilde{R}_{i+1,t}^k, \tilde{R}_{i,t}^k \in \{R_{i,t}^k\}_{i=1}^N \quad (43)$$

The aircraft are sent to maintenance check according to this sorted list while aircraft with a lower remaining utilization are given a higher check priority. Since the C-check is more restrictive and demanding in terms of resources, it has higher priority than an A-check. In addition to the available slots and maintenance elapsed time of the check type, we set the following rules for making A- and C-check decisions:

- (i) No type k check should be scheduled if there is no available hangar for type k check on day t .
- (ii) An aircraft is allocated a type k check if it has the lowest remaining utilization $\tilde{R}_{1,t}^k = \min \{R_{i,t}^k\}$ and there are available slots for type k check $\left(\exists h, M_{h,\tau}^k > 0 \text{ for } \forall \tau \in [t, t + L_i^k(y_{i,t}^k) - 1] \right)$.

- (iii) The A-check of aircraft i can be incorporated into C-check if $\delta_{i,t}^C = 1$ and $L_i^A(y_{i,t}^A) < L_i^C(y_{i,t}^C)$.
- (iv) If aircraft i has a higher type k check priority than aircraft j ($\tilde{R}_{i,t}^k < \tilde{R}_{j,t}^k$) but the remaining slots of type k check are only sufficient to accommodate a type k check for aircraft j rather than i , swap the priorities of type k check between aircraft i and j .

After assigning A- and C-check priorities to each aircraft, the combination of aircraft selection for maintenance, as well as the number of outcome states, is reduced from $\frac{(N!)^2}{m_C!(N-m_C)!m_A!(N-m_A)!}$ to 1.

4.3. Thrifty Algorithm for A- and C-Check Scheduling

Even after reducing the size of outcome space to one action per state, the number of final states n_{act}^{T+1} is very tremendous for a large T , we need to further trim the outcome space so that forward induction is tractable.

After an action is performed, we place ourselves at state s_{t+1} , where $s_{t+1} = \mathcal{S}^{\mathcal{X}}(s_t, x_t)$. Many s_{t+1} states may have the status that some aircraft will have to be grounded to wait for a A-/C-check slot in a future stage τ ($\tau > t + 1$). Apparently this is what airlines to avoid (unless they have no better option), due to the very high cost of parking aircraft on the ground. Therefore, we propose to only consider the actions that lead to a *workable* state. We describe *workability* of s_{t+1} by using the following function:

$$g(s_{t+1}) = \sum_{k \in \{A, C\}} \sum_{i=1}^N \sum_{\tau=t+1}^T \eta_{i,\tau}^k \quad (44)$$

A state s_{t+1} , resulting from being at state s_t and taking action x_t , is said to be *workable* if there exists a sequence of actions $x_{t+1}, \dots, x_T$ such that no aircraft has to wait on the ground for an A- or C-check between $t + 1$ and T . That is:

$$g(s_{t+1}) = 0 \quad (45)$$

In our approach, we use a *Thrifty Algorithm* to check the *workability* of future states (**Block B** in Figure 2). That is, for each possible action x_t and resulting state s_{t+1} , we use an algorithm to check if a sequence of actions exist that guarantee (45). If that is the case, we consider s_{t+1} to be *workable* and otherwise we assume that x_t does not lead to such a *workable* state.

The term *Thrifty* refers to the concept that we make use of all available slots, allocating A- and C-check to aircraft whenever there is a maintenance opportunity. The *Thrifty Algorithm* serves the purpose of checking the workability of a state s_t . For convenience, in the rest of the paper, we refer to running the *Thrifty Algorithm* to check whether (45) holds when mentioning “checking *workability*”.

4.4. Discretization and State Aggregation

After moving one stage ahead in time for a set of *workable* states s_t , several *workable* s_{t+1} states are produced from a combination of s_t and x_t . We use S_{t+1} to denote the set of *workable* s_{t+1} :

$$S_{t+1} = \{s_{t+1} | s_{t+1} \text{ workable}\} \quad (46)$$

Although the *Thrifty Algorithm* can help reduce the outcome space to a certain extent by only keeping the *workable* s_{t+1} , the number of *workable* s_{t+1} is still not bounded; meaning that the number of *workable* states may still grow exponentially. This increases the difficulty of saving all *workable* s_{t+1} and tracing the actions backwards, especially if we move several stages ahead. In order to prevent the explosion of *workable* states, we need to restrain the number of *workable* s_{t+1} , from the first stage t_0 to final stage T . That is, giving an upper bound to the number of *workable* s_{t+1} so that it will not increase exponentially with increasing t . For such purpose we resort to *discretization and state aggregation*.

Discretization is the process of transferring continuous models or variables into discrete counterparts. *State aggregation* refers to collecting or clustering the states that have the same properties into a group. Here we use ‘properties’ to differentiate state attributes, which are the features that at a fleet level, such as mean utilization of fleet (A- or C-check) or standard deviation of fleet utilization (A- or C-check). We divide the outcome space (a set of *workable* s_{t+1}) into several disjunct space regions, where each space region is characterized by a unique tuple of values of some state properties. For the states that clustered in the same space region (having the same tuple of state properties), only one single state will be selected to represent such a space region and considered in forward induction for the next stage.

Such *discretization and state aggregation* provides an upper bound to the outcome space, since the number of *workable* s_{t+1} is determined by the number of space regions. One way of collecting *workable* states is to discretize the AMCS problem according to the mean utilization of the fleet with respect to A- and C-checks, and then categorize the *workable* states according to the values of the features:

$$\bar{u}_{t+1}^A = \frac{1}{N} \sum_{i=1}^N u_{i,t+1}^A, \quad \bar{u}_{t+1}^C = \frac{1}{N} \sum_{i=1}^N u_{i,t+1}^C \quad (47)$$

where $u_{i,t+1}^k$ is the utilization of aircraft i with respect to check type k ($k \in \{A, C\}$). We define individual utilization $u_{i,t}^k$ as the maximum of the ratios between the current value of the usage parameters and their respective maximum values, according to the MPD:

$$u_{i,t+1}^k = \max \left\{ \frac{DY_{i,t+1}^k}{I_{k-DY}^i - \epsilon_{i,t+1}^{k-DY}}, \frac{FH_{i,t+1}^k}{I_{k-FH}^i - \epsilon_{i,t+1}^{k-FH}}, \frac{FC_{i,t+1}^k}{I_{k-FC}^i - \epsilon_{i,t+1}^{k-FC}} \right\} \quad (48)$$

for $k \in \{A, C\}$. For each check type k , we also give upper bound $U_{\max}^k$ and lower bound $U_{\min}^k$ to restrict the outcome space region to be discretized. This significantly improves algorithm efficiency and reduces required computer memory when optimizing A- and C-check schedules for a large fleet. For instance, if a fleet has about 200 aircraft, performing A- or C-check on an aircraft will only impact the overall fleet utilization slightly. In such case, $U_{\max}^k$ and $U_{\min}^k$ can be chosen close to $\bar{u}_{t_0}^k$.

Since tolerance is not allowed in planning unless no feasible schedule can be found, the mean utilization of a fleet normally ranges between 0 and 1 in practice ($U_{\max}^k = 1$ and $U_{\min}^k = 0$), a discretization increment $\Delta u = 0.1$ yields 11^2 space regions in 1 stage, while an increment of $\Delta u = 0.01$ increases the number of space regions in 1 stage increase to 101^2 .

Using $\bar{u}_{t+1}^A$ and $\bar{u}_{t+1}^C$ to categorize the set of *workable* s_{t+1} enables us to cover the state properties of

both the A- and C-check, with each pair $(\bar{u}_{t+1}^A, \bar{u}_{t+1}^C)$ corresponding to one outcome space region. For each workable state s_{t+1} , we compute the mean fleet utilization with respect to the A- and C-check, $\bar{u}_{t+1}^A$ and $\bar{u}_{t+1}^C$, from (47) and (48). These two features will be further rounded according to the number of decimal points chosen from Δu . For example, if $\Delta u = 0.1$, $\bar{u}_{t+1}^C = 0.345$ and $\bar{u}_{t+1}^A = 0.678$, then $\bar{u}_{t+1}^C$ can be rounded to 0.3 and $\bar{u}_{t+1}^A$ to 0.7. After that, we compute the cumulative contribution from state s_{t_0} to a specific *workable* state s_{t+1} :

$$J_{t+1, \bar{u}_{t+1}^A, \bar{u}_{t+1}^C}(s_{t+1}) = \begin{cases} J_{t, \bar{u}_t^A, \bar{u}_t^C}(s_t) + C_t(s_t, x_t) & t > t_0 \\ C_{t_0}(s_{t_0}, x_{t_0}) & t = t_0 \end{cases} \quad (49)$$

where $s_{t+1} = \mathcal{S}^{\mathcal{X}}(s_t, x_t)$, $C_t(s_t, x_t)$ refers to the contribution function in (32). If a given space region has no state within it, a cumulative contribution value of infinity ∞ is assumed for that space region.

During forward induction, there will be several workable states that can be grouped into the same outcome space region because of identical $\bar{u}_{t+1}^A$ and $\bar{u}_{t+1}^C$ after rounding. Then, an aggregation procedure is followed: the state with the lowest cumulative contribution is selected as the representative of its outcome space region, while all others are discarded:

$$s_{t+1}^* = \underset{s}{\operatorname{argmin}} \left\{ J_{t+1, \bar{u}_{t+1}^A, \bar{u}_{t+1}^C}(s) \right\} \quad (50)$$

In the worst case, no s_t has a subsequent workable s_{t+1} , that is, $g(s_{t+1}) = g(\mathcal{S}^{\mathcal{X}}(s_t, x_t)) > 0$ for all s_t and $x_t = \mathcal{X}^{\pi}(s_t)$. In such a circumstance, we select only one $\hat{s}_{t+1}$ according to:

$$\hat{s}_{t+1} = \underset{s, \bar{u}_t^A, \bar{u}_t^C}{\operatorname{argmin}} \left\{ J_{t, \bar{u}_t^A, \bar{u}_t^C}(s) \right\} \quad (51)$$

$$S_{t+1} = \{\hat{s}_{t+1}\} \quad (52)$$

where the right hand side of (51) means choosing the state s among all outcome space regions $(\bar{u}_{t+1}^A, \bar{u}_{t+1}^C)$. The forward induction then continues from $\hat{s}_{t+1}$.

The procedure from (49) to (50) repeats until it loops all possible pairs of $\{s_t, x_t\}$ (s_t *workable*). Thus far, we complete the *discretization and state aggregation*, and then the forward induction moves one stage ahead from t to $t + 1$. The pseudo code of DP based methodology is presented in [Appendix B](#).

4.5. Algorithm Complexity

From the perspective of algorithm complexity, the total number of states in our DP based methodology is equivalent to the total number of outcomes, given by (37):

$$\prod_{t=t_0}^T \sum_{m_c=0}^{M_t^C} \sum_{m_a=0}^{M_t^A} \frac{(N!)^2}{m_a! m_c! (N - m_a)! (N - m_c)!} \quad (53)$$

where N is the total number of aircraft, and M_t^C and M_t^A are the maintenance capacity of C-check and A-check respectively.

Given a state s_t at time stage t , following an action x_t , the algorithm has to call the state transition function (6) at most $T - t_0 - t + 1$ times to check whether (45) holds (from t to T). In each stage t , the number of states depends on the discretization level Δu :

$$n_{\text{state}} = \left(1 + \frac{1}{\Delta u}\right)^2 \quad (54)$$

Since each state can have at most n_{act} actions, this implies the following relation between the stage t and the number of state transition:

$$\begin{array}{ll} \text{Day } t_0: & n_{\text{act}} (1 + T - t_0) \\ \text{Day } t_0 + 1: & n_{\text{state}} n_{\text{act}} (1 + T - 1 - t_0) \\ \dots & \dots \\ \text{Day } T - 1 & n_{\text{state}} n_{\text{act}} (1 + T - T) \\ \text{Day } T & n_{\text{state}} n_{\text{act}} (1 + T - T - 1) \end{array}$$

After summing up of all state transitions from t_0 to T we obtain

$$\begin{aligned} & n_{\text{act}} (1 + T - t_0) + n_{\text{state}} n_{\text{act}} \sum_{\tau=0}^{T-t_0} \tau \\ &= n_{\text{act}} (1 + T - t_0) \left[1 + \frac{n_{\text{state}} (T - t_0)}{2} \right] \\ &= n_{\text{act}} (1 + T - t_0) \left[1 + \frac{1}{2} \left(1 + \frac{1}{\Delta u} \right)^2 (T - t_0) \right] \end{aligned} \quad (55)$$

Computing (55) gives the maximum number of state transitions in forward induction. Since each of the state transitions generates a new state, this means that the total number of states to be visited is equal to the total number of state transitions in forward induction.

Moreover, (55) also indicates that the total number of states visited during forward induction depends on the A- and C-check capacity (n_{act} is determined by the A- and C-check capacity M_t^A and M_t^C), and the increment of discretization Δu and planning horizon $T - t_0$. Since (55) increases quadratically with T , this means that (55) can be much smaller than (53) for a large T :

$$n_{\text{act}} (1 + T - t_0) \left[1 + \frac{1}{2} \left(1 + \frac{1}{\Delta u} \right)^2 (T - t_0) \right] << \prod_{t=t_0}^T \sum_{m_c=0}^{M_t^C} \sum_{m_a=0}^{M_t^A} \frac{(N!)^2}{m_a! m_c! (N - m_a)! (N - m_c)!} \quad (56)$$

5. Case Study

In this section, the proposed DP based methodology is evaluated using the aircraft maintenance data from a European airline. Two case studies are presented in this evaluation. The first cases uses data from the historical period 2013 to 2016 to validate the proposed DP based methodology. We compare the results

Table 2: A- and C-check intervals and tolerance for the Airbus A319, A320 and A321 [28].

	Check Type	Calendar Days	Flight Hours	Flight Cycles
Inspection intervals	A-Check	120	750	750
	C-Check	730	7500	5000
Tolerance	A-Check	12	75	75
	C-Check	60	500	250

obtained by the DP based methodology with the A- and C-check schedule executed by the airline. The second case focuses on the period of 2018-2021 and compares the results from the DP based methodology with the maintenance schedule planned by the airline, and supports a sensitivity analysis on some of the model parameters.

5.1. Test Cases

The test fleet is the Airbus A320 family (A319, A320 and A321) operated by the airline, consisting of 45 aircraft. These three aircraft types happen to share the same A- and C-check intervals and tolerances, in terms of same flight hours, flight cycles and calendar days (Table 2). The planning horizon is 4 years in both cases. For 2013-2016, this starts from January 1st of 2013 to December 31st of 2016, while 2018-2021 the planning horizon goes from September 25th of 2017 to December 31st of 2021. For both cases the initial data contains the information for aircraft average monthly utilization; initial fleet status, in terms of DY, FH and FC, and utilization of tolerance in previous inspections; maintenance slots available per day; and average elapsed time of the multiple A- and C-checks labels. The average daily utilization of the aircraft is computed per month and per aircraft type, according to the historic flight data from the airline. On average, it is estimated that the A320 family fleet has a daily utilization of 10.5 FH and 4.7 FC per day.

5.2. Maintenance Constraints and Key Performance Indicators

The maintenance schedule needs to follow a set of operational and capacity constraints, namely, for the A-check:

- there is 1 A-check slot per day from Monday to Thursday during IATA winter (from the last Sunday of October to the last Sunday of March);
- during IATA Summer (from the last Sunday of March to last Sunday of October), there is an extra A-check slot on Tuesdays (2 slots on Tuesday);
- from 2018 onwards, there are 2 A-check slots on Tuesdays (all year) and 2 A-check slot on Wednesdays during IATA Summer;
- there are no A-checks on Fridays, weekends, or public holidays;

- an A-check lasts 1 day and can be merged into a C-check without increasing the C-check elapsed time or affecting the existing available A-check slots.

For the C-check:

- there can be a maximum of 3 C-checks ongoing in parallel;
- there are a minimum of 3 days between the start dates of two C-checks, for resource availability reasons (i.e., $d_C = 3$);
- C-check works are interrupted during weekends and public holidays;
- no C-check can be scheduled during the commercial peak seasons (except some extraordinary occasions in which the airline is forced to have additional slots to avoid aircraft waiting on the ground for a C-check).

Since there are at least 3 days between two start dates of two successive C-checks, there could be at most 1 C-check starting on a day. The maximum of 2 A-checks on a Tuesday and considering the possibility of merging A-checks into C-check, leads to the combination of daily A- and C-check capacities with 7 possible actions on a day:

- (i) 0 A-check & 0 C-check
- (ii) 0 A-check & 1 C-check
- (iii) 1 A-check & 0 C-check
- (iv) 1 A-check & 1 C-check
- (v) 2 A-checks & 0 C-check
- (vi) 2 A-checks & 1 C-check
- (vii) 3 A-checks & 1 C-check

The commercial peak seasons of the airline are defined to be between June 1st and September 30th, two weeks before the New Year's and one week after, and the weeks before and after Easter. The days of the year are converted into calendar days where, e.g., the New Year's Day is set as day 1 and Christmas is day 359 of the year (or day 360 if it is a leap year).

To discuss the results, we use a set of key performance indicators (KPIs) for each type of maintenance check. These are the average FH of the fleet, the total number of checks, the total amount DY/FH/FC used as tolerance and computation time etc during the planning horizon. For deterministic problems, we make the transition probability $p(s_{t+1}|s_t) = 1$ in (35). Since no information is given for discount factor γ , we set $\gamma = 1$ and the penalty of using tolerance P_a and P_d in (32) are given to be 10^8 . This avoids using tolerance in forward induction and grounding the aircraft in the situation of no A- or C-check slot.

Table 3: Descriptive statistics of KPIs for 2013-2016 ($\Delta u = 0.1$ in the DP based method). For the term “Tolerance Events”, if an aircraft uses tolerance (DY/FH/FC) in planning, it is counted as 1 tolerance event.

Type	KPI 2013-2016	Airline	DP-based Method.	Difference
C-Check	Average FH	6795.9	6798.7	0.04%
	Total FH Tolerance	2230	349.2	-84.3%
	Tolerance Events	15	9	-40%
	Extra C-Check Slot	73	0	-100%
	Total C-Check	89	82	-7.9%
A-Check	Average FH	690.8	701.2	1.5%
	Total FH Tolerance	1277	457.4	-64.2%
	Tolerance Events	82	59	-28.0%
	Extra A-Check Slots	101	0	-100%
	Total A-Check	818	758	-7.3%

5.3. Optimization Results for 2013-2016

The proposed algorithm is first evaluated for the planning horizon of 2013-2016. Table 3 shows a comparison of KPIs between the airline schedule and the DP schedule. We observe that the average FH increases with respect to both A- and C-checks. For the A-check, there is a growth of 10.4 FH on average per aircraft, which equates to approximate an extra day in operation per aircraft per A-check cycle. This increase has an impact on the number of checks needed for the 4-year period. There is a reduction of more than 7% for both A- and C-checks, which is equivalent to 60 fewer A-checks and 7 fewer C-checks. From the perspective of maintenance cost, assuming that airlines spend on average \$70K—\$350K [2] on a C-check and \$10K—\$15K on a A-check, the results from the proposed DP-based methodology can result in a maintenance costs saving of approximate \$1.1M—\$3.4M for the fleet of A320 family.

Since it takes 10-30 days to complete a C-check and 1 day for a A-check, 7 reduced C-checks and 60 reduced A-checks are equivalent to approximately 130—270 more days of aircraft availability for revenue generation. One day of operation generates \$75K—\$120K of revenue and 130—270 more days available for commercial use means an additional \$9.8M—\$32.4M of revenue for an airline.

The optimized schedule uses tolerances of 349.2 FH and 457.4 for the A- and C-check scheduling, respectively. These are 84% and 64% less than the FH tolerances used by airline. More importantly, the optimized schedule reduces the frequency of using tolerance (if an aircraft uses tolerance, it is counted as 1 tolerance events), from 15 to 9 with respect to C-check, and from 82 to 59 for the A-check. Recall that using tolerance needs to be approved by the national aviation authority and it is a troublesome process that should not be used recurrently.

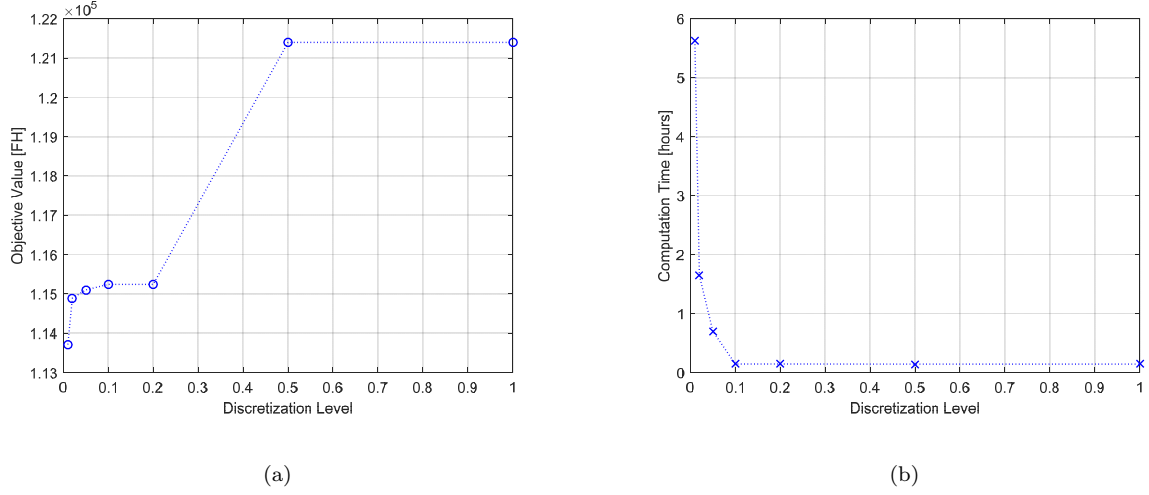


Figure 3: (a) correlation between discretization level and objective value; (b) correlation between discretization level and algorithm computation time.

5.4. Optimization Results for 2018-2021

Next we use the proposed methodology to subsequently optimize the airline A- and C-check maintenance schedule for future 2018-2021. We compare the optimization results of different discretization levels ($\Delta u = 1, 0.1$ and 0.01) with the current C-check schedule from the airline. Given that the airline only plans the A-check for the coming year, no A-check metrics were compared. The optimized schedules of different discretization levels (both A- and C-checks use the same Δu in discretization) are obtained using parallel computing function on a quad-core workstation.

An overall trend is found where the level of discretization impacts the optimality and algorithm computation time, as illustrated in Figure 3a and 3b. In terms of optimality, as expected, the smaller Δu is, the better the results are. However, the KPI's presented in Table 4 are no significantly different between the three Δu values tested, indicating that good results can be obtained even with low discretization levels. This happens because most *workable* states are in a limited range of our outcome space. Just a few of the space regions from our discretization do really have a *workable* state after aggregation. Nevertheless, the objective function consistently reduces when we increase the discretization level while the computational times sharply increase when we reduce the discretization level beyond $\Delta u = 0.1$.

Compared with the C-check schedule of an airline, the optimized schedules reduce the number of C-checks, varying from 8.3% (for $\Delta u = 0.1$) to 11.4% (for $\Delta u = 0.01$), while the same amount of tolerance is used in all three optimized schedules. The tolerance and the number of tolerance events from our results is significantly less than what the airline scheduled. The use of this tolerance is inevitable for aircraft that at the starting date of the optimization are already closed to their C-check interval. The number of A-checks vary from 877 (for $\Delta u = 0.1$) and 881 (for $\Delta u = 1$ and $\Delta u = 0.01$), when the airline estimates around 895 to 920

Table 4: Results of A- and C-check scheduling optimization from different discretization level, compared with the C-check schedule from Airline.

KPI (2018-2021)	Airline	$\Delta u = 1$	$\Delta u = 0.1$	$\Delta u = 0.01$
Objective Value [FH]	—	1.2140×10^5	1.1524×10^5	1.1371×10^5
C-Check Avg. FH	< 6600	6558.1	6615.2	6634.7
Total C-Checks	96	86	88	85
C-Check Tol. DY	> 48	18	18	18
C-Check Tol. FH	> 490	135.3	135.3	135.3
C-Check Tol. FC	0	0	0	0
Tolerance Events	6	4	4	4
A-Check Avg. FH	—	714.3	717.6	714.5
Total A-Checks	895-920 *	881	877	881
Tolerance Events	—	0	0	0
Merged A- and C-Check	—	19	18	19
Computation Time [s]	—	504.9	510.3	20243.5

*Airline estimation

A-checks for these four years. The number of A-checks merged in the C-checks has little variance among three discretization levels.

In terms of algorithm computation time, decreasing Δu results in longer computation times. This happens because the number of outcome space region increases with decreasing Δu , e.g., there are 121 outcome space regions when $\Delta u = 0.1$, and 10201 regions for $\Delta u = 0.01$. As a result, the number of representative states of outcome space region in each stage also increases. However, for this particular case study, the computational time only starts to sharply increase at Δu values of lower than 0.1. After breaking down the computation time, we observe that each state transition requires about 0.023s. Looking at the case of $\Delta u = 0.1$, it takes 510.3s to obtain an optimized schedule using parallel computing on a quad-core workstation, meaning that the actual computation time should be about 2141.2s. In particular, during 2141.2s, there are about 2141.2/0.023 = 9.31×10^4 state transitions for 1461 stages (there being 1461 days from Jan 1st 2018 to Dec 31st 2021). The actual computation is still much less than the worst case computed from (55):

$$\begin{aligned}
\text{computation time} &= 0.023 \times n_{\text{act}} (1 + T - t_0) \left[1 + \frac{1}{2} \left(1 + \frac{1}{\Delta u} \right)^2 (T - t_0) \right] \\
&= 0.023 \times 7 \times 1461 \times \left[1 + \frac{1}{2} \times 11^2 \times 1460 \right] \approx 2.08 \times 10^7 \text{ (s)}
\end{aligned} \tag{57}$$

The reason for shorter actual computation time than the worst case is due to the checking of *workability* and *stage aggregation*, where we only keep some *workable* states in each stage, which in the end are sufficient to generate an optimized schedule.

Table 5: Sensitivity analysis for having different A- and C-check slots in 2018-2021, the discretization level is set to $\Delta u = 0.1$. No A-check tolerance was used in the scenarios tested.

KPI (2018-2021)	Scenario 0	Scenario 1	Scenario 2	Scenario 3
Objective Value [FH]	1.1524×10^5	0.8934×10^5	1.0623×10^5	0.7719×10^5
C-Check Avg. FH	6615.2	6635.0	6699.3	6790.5
Total C-Checks	88	85	85	86
C-Check Tol. DY	18	18	0	0
C-Check Tol. FH	135.3	135.3	23.6	23.6
C-Check Tol. FC	0	0	0	0
A-Check Avg. FH	717.6	738.5	716.5	739.8
Total A-Checks	877	856	890	863
Merge A- and C-check	18	7	22	10
Computation Time [s]	505.6	1780.2	743.0	2625.5

5.5. Sensitivity Analysis for 2018-2021

This subsection investigates the impact of some airline capacity constraints on the results of the AMCS problem, relative to the following 4 scenarios:

- *Scenario 0*: the baseline scenario, as pre-computed in the previous subsection;
- *Scenario 1*: conditions from Scenario 0 and one additional A-check slot on Friday, weekends and bank holidays (i.e., one A-check slot every day of the week, plus an extra slot on Tuesdays and Wednesdays during IATA Summer);
- *Scenario 2*: conditions from Scenario 0 and three additional C-checks on weekends and bank holidays (i.e., three C-check slots every day of the week during off peak seasons, reducing the elapse time of the C-checks);
- *Scenario 3*: conditions from Scenario 0, Scenario 1, and Scenario 2 combined.

All four scenarios are computed assuming $\Delta u = 0.1$, relative to the sensitivity shown in Table 4 regarding precision and computation time. The results, shown in Table 5, indicate that a natural improvement of the aircraft average utilization is obtained when either the A-check or the C-check slots are increased. By increasing the number of checks, we are increasing the maintenance opportunities, given more flexibility for a schedule in which the checks are planned closer to their due date. For example, compared with *Scenario 0*, the objective value in *Scenario 3* is reduced by 33%, and the average FH of aircraft is increased by 2.7% and 3.1% for C-check and A-check, respectively. As consequence of this, there are 2 fewer C-checks and 14 fewer A-checks scheduled for *Scenario 3* when compared with *Scenario 0*. For the scenarios involving more C-check slots, it is observed that the results are improved by the fact that more maintenance opportunities exist to merge A-checks and C-checks. In fact, *Scenarios 2* and *3* have around 500 to 600 fewer days on the ground

than *Scenarios 1* and *2* respectively. These are days when the aircraft can be used in operation to generate revenue. Furthermore, it is interesting to notice the interdependence between A- and C-checks when analyzing the results from *Scenario 1*. Although only additional A-check slots are added, the results for the C-checks also improve, due to the fact that more A-check slots create more A-check maintenance opportunities. This gives more flexibility to schedule some of the C-checks that in *Scenario 0* were anticipated to enable the merge with an A-check.

The consideration of extra aircraft maintenance capacity has to be analyzed by the airline by comparing the additional costs of these extra slots and the benefits of having less maintenance checks and higher aircraft availability. This analysis is outside the scope of this paper but these results are crucial to the airline in assessing such capacity increases (or reductions) scenarios.

6. Conclusion

A practical dynamic programming based methodology for the long-term aircraft maintenance check scheduling (AMCS) problem is presented. This integrates both A- and C-checks, including requirements that are associated with the previous denominated B- and D-checks, operational constraints and maintenance capacity for specific days. The goal was to minimize the total wasted FH interval between checks, hereby increasing aircraft availability in the long run.

The proposed methodology followed a *forward induction* approach, incorporating a maintenance *priority solution* to deal with the multi-dimensional action vector, as well as a *discretization and state aggregation* strategy to reduce outcome space at each time stage. In addition, a *Thrifty Algorithm* was used to estimate the consequence of an action at the current stage on the remaining planning horizon. The proposed methodology is capable of optimizing both A- and C-check schedules in a matter of minutes for multiple years horizon and heterogeneous aircraft fleets. It is suitable for practical implementation and it can be used not only for scheduling but also, for example, to predict if an airline has sufficient maintenance capacity in the future; or to assess if it is beneficial to expand maintenance capacity with additional hangar slots.

The proposed DP based methodology is evaluated using the case-study of an A320 family fleet from a European airline. Comparing the optimized A- and C-check schedules with the schedule of the airline, we can infer that the proposed methodology reduces the total number of A- and C-check, potentially resulting in the long run in maintenance cost savings of about \$1.1M-\$3.4M for a fleet of about 40 aircraft. Besides, the reduction of A- and C-checks implies extra days of aircraft availability for revenue generation, an estimation of \$9.8M-\$32.4M can be generated when the proposed methodology is applied on historical data.

This study is the first to address the AMCS optimization problem despite its relevance for practice. It opens the door for future research on the topic. For instance, future research can consider the uncertainty associated with both the maintenance check elapse times and the aircraft utilization. These uncertainties will not only affect the schedule robustness, but also the computational time needed to compute such optimal schedules. One such improvement can be achieved by using approximate dynamic programming, extending

the dynamic programming principle adopted in this paper. Another research opportunity, is the consideration of the task packaging problem (i.e., the problem of defining the tasks to be performed on each aircraft check) as part of the AMCS problem. Although this would significantly increase the complexity of the problem, it would extend the AMCS problem to good benefit, producing an optimal integrated check and tasks schedule.

Acknowledgments

This research work is part of AIRMES project, which received funding from the Clean Sky 2 Joint Undertaking under the European Union's Horizon 2020 research and innovation programme under grant agreement No 681858. We would like to show gratitude to the associated AIRMES partners for providing their aircraft maintenance data and validation of the proposed methodology and its practical relevance. For more information please visit www.airmes-project.eu.

References

- [1] Canadian Aviation Regulations 2012-1, Part I - General Provisions, Subpart 1 - Interpretation (December 2012).
URL <https://web.archive.org/web/20121227092905/http://www.tc.gc.ca:80/eng/civilaviation/regserv/cars/part1-subpart1-1104.htm>
- [2] S. P. Ackert, Basics of aircraft maintenance programs for financiers, Evaluation and Insights of Commercial Aircraft Maintenance Programs.
URL http://aircraftmonitor.com/uploads/1/5/9/9/15993320/basics_of_aircraft_maintenance_programs_for_financiers___v1.pdf
- [3] C. V. Buskirk, B. Dawant, G. Karsai, J. Sprinkle, G. Szokoli, K. Suwanmongkol, R. Curren, Computer-Aided Aircraft Maintenance Scheduling, Tech. rep., Institute for Software-Integrated Systems (2002).
- [4] P. Horder, Airline Operating Costs, in: Managing Aircraft Maintenance Costs Conference, 2003.
- [5] IATA's Maintenance Cost Task Force, Airline Maintenance Cost Executive Commentary, Tech. rep. (2015).
- [6] A. Steiner, A Heuristic Method for Aircraft Maintenance Scheduling under Various Constraints, in: 6th Swiss Transport Research Conference, 2006.
- [7] N. J. Boere, Air Canada Saves with Aircraft Maintenance Scheduling, Interfaces 7 (3) (1977) 1–13.
- [8] M. Etschmaier, P. Franke, Long-Term Scheduling of Aircraft Overhauls, in: AGIFORS Symposium, 1969.
- [9] H. Bauer-Stämpfli, Near Optimal Long-Term Scheduling of Aircraft Overhauls by Dynamic Programming, in: AGIFORS Symposium, 1971.
- [10] P. Belobaba, A. Odoni, C. Barnhart, Global Airline Industry, John Wiley and Sons, 2009.
- [11] T. A. Feo, J. F. Bard, Flight Scheduling and Maintenance Base Planning, Management Science 35 (12) (1989) 1415–1432.
- [12] W. E. Moudani, F. Mora-Camino, A Dynamic Approach for Aircraft Assignment and Maintenance Scheduling by Airlines, Journal of Air Transport Management 6 (2000) 233–237.
- [13] N. Papakostas, P. Papachatzakis, V. Xanthakis, D. Mourtzis, G. Chrysosouris, An approach to operational aircraft maintenance planning, Decision Support Systems 48 (2010) 604–612.
- [14] M. Başdere, U. Bilge, Operational aircraft maintenance routing problem with remaining time consideration, European Journal of Operational Research 235 (2014) 315–328.

- [15] Z. Liang, Y. Feng, X. Zhang, T. Wu, W. A. Chaovaitwongse, Robust weekly aircraft maintenance routing problem and the extension to the tail assignment problem, *Transportation Research Part B* 78 (2015) 238–259.
- [16] C. Sriram, A. Haghani, An Optimization Model for Aircraft Maintenance Scheduling and Re-Assignment, *Transportation Research Part A* 37 (2003) 29–48.
- [17] C. Senturk, M. S. Kavsoglu, M. Nikbay, Optimization of Aircraft Utilization by Reducing Scheduled Maintenance Downtime , in: 10th AIAA Aviation Technology, Integration, and Operations (ATIO) Conference, 2010.
- [18] H. A. Kinnison, T. Siddiqui, McGraw-Hill Education, 2012.
- [19] C. Senturk, I. Ozkol, The Effects of the Use of Single Task-Oriented Maintenance Concept and More Accurate Letter Check Alternatives on the Reduction of Scheduled Maintenance Downtime of Aircraft , *International Journal of Mechanical Engineering and Robotics Research*.
- [20] S. Duffuaa, K. Al-Sultan, Mathematical programming approaches for the management of maintenance planning and scheduling, *Journal of Quality in Maintenance Engineering* 3 (3) (1997) 163–176.
- [21] S. Dreyffus, Richard Bellman on the Birth of Dynamic Programming, *Operations Research* 50 (1).
- [22] E. E. Bomberger, A Dynamic Programming Approach to a Lot Size Scheduling Problem, *Management Science* 12 (11) (1966) 778–784.
- [23] A. Gascon, R. C. Leachman, A Dynamic Programming Solution to the Dynamic, Multi-Item Single-Machine Scheduling Problem, *Operations Research* 36 (1) (1988) 50–56.
- [24] E. L. Lawler, A Dynamic Programming Algorithm for Preemptive Schedule of a Single Machine to Minimize the Number of Late Jobs, *Annals of Operations Research* 26 (1) (1990) 125–133.
- [25] G. H. Graves, C.-Y. Lee, Scheduling maintenance and semiresumable jobs on a single machine, *Naval Research and Logistics* 46 (7) (1999) 845–863.
- [26] M. Pereira, N. Campodónico, R. Kelmam, Long-term Hydro Scheduling based on Stochastic Models, *EPSOM* 39 (1998) 1170–1190.
- [27] T. Asamov, D. F. Salas, W. B. Powell, [SDDP vs. ADP: The Effect of Dimensionality in Multistage Stochastic Optimization for Grid Level Energy Storage](http://asamov.com/download/SDDP-ADP.pdf).
URL <http://asamov.com/download/SDDP-ADP.pdf>
- [28] Airbus A320 Maintenance Planning Document [Private Document].
- [29] W. B. Powell, *Approximate Dynamic Programming - Solving the Curses of Dimensionality*, 2011.

- [30] R. H. Hoppe, [Optimization Theory II - Lecture Notes](https://www.math.uh.edu/~rohop/Spring_12/index.html), 2018.
URL https://www.math.uh.edu/~rohop/Spring_12/index.html

Appendix A. Nomenclature

Parameters:

d_k	Minimum interval between the start dates of two type k checks.
e_{k-DY}^i	Maximum DY tolerance of type k check interval of aircraft i
e_{k-FH}^i	Maximum FH tolerance of type k check interval of aircraft i
e_{k-FC}^i	Maximum FC tolerance of type k check interval of aircraft i
$fc_{i,t}$	Average daily FC usage for aircraft i at day t
$fh_{i,t}$	Average daily FH usage for aircraft i at day t
h_A	A-check hangar indicator
h_C	C-check hangar indicator
i	Aircraft indicator
I_{k-DY}^i	Interval of type k check of aircraft i in terms of DY
I_{k-FH}^i	Interval of type k check of aircraft i in terms of FH
I_{k-FC}^i	Interval of type k check of aircraft i in terms of FC
k	Maintenance check type, $k \in \{A, C\}$
N	Total number of aircraft
n_k	The number of hangars for type k check
n_{act}	The number of actions on day t
P_d	Daily penalty for having an aircraft on the ground waiting for a maintenance slot
P_a	Penalty for an aircraft using the tolerance
t	Indicator of calendar day
T	Final day in planning horizon
t_0	First day in planning horizon
Δu	Increment of fleet utilization for discretization
π	Scheduling policy
γ	Discount factor

Decision Variables:

$\chi_{i,t}^k$	Binary variable to indicate if aircraft i starts type k check on t
x_t	Available action on day t , $x_t = \{\{\chi_{i,t}^C\}, \{\chi_{i,t}^A\}\}$
x_t^*	The optimal action among $\{x_t\}$
$\mathcal{X}^\pi(s_t)$	Scheduling policy function, $x_t = \mathcal{X}^\pi(s_t)$

State Related Decision Variables:

$a_{i,t}$	The attributes of aircraft i in the beginning of day t
A_t	$A_t = \{a_{i,t} \mid i = 1, 2, \dots, N\}$
$C_t(s_t, x_t)$	Contribution of choosing action x_t on s_t
$DY_{i,t}^k$	Total DY of aircraft i in the beginning of day t for type k check
$FC_{i,t}^k$	Cumulative FC of aircraft i at t since last type k check
$FH_{i,t}^k$	Cumulative FH of aircraft i at t for type k check
$J_{t, \bar{u}_t^A, \bar{u}_t^C}(s_t)$	Cumulative contribution on day t when the fleet has mean utilization $\bar{u}_t^A$ and $\bar{u}_t^C$ for A-check and C-check respectively
$J_{t, \bar{u}_t^A, \bar{u}_t^C}^{\min}(s_t)$	$J_{t, \bar{u}_t^A, \bar{u}_t^C}^{\min}(s_t) = \min \left\{ J_{t, \bar{u}_t^A, \bar{u}_t^C}(s_t) \right\}$
$L_i^k(y_{i,t}^k)$	Estimated elapsed time of next type k check with label $y_{i,t}^k$
$M_{h,t}^k$	Binary variable to indicate if type k check can be performed in hangar h_k on day t
M_t^k	Hangar capacity of type k check, $M_t^k = \sum_h M_{h,t}^k$
s_t	State variable
S_t	The set of workable states, $S_t = \{s_t \mid s_t \text{ workable}\}$
$R_{i,t}^k$	Remaining fly days of aircraft i before the next type k check
$y_{i,t}^k$	Next maintenance label for of type k check of aircraft i on day t
$z_{i,t}^k$	The end date of type k check of aircraft i
$\delta_{i,t}^k$	Binary variable to indicate if aircraft i is undergoing type k check on day t
$\epsilon_{i,t}^{k-DY}$	Extra DY before day t if previous type k check is deferred
$\epsilon_{i,t}^{k-FH}$	Extra FH before day t if previous type k check is deferred
$\epsilon_{i,t}^{k-FC}$	Extra FC before day t if previous type k check is deferred
$\eta_{i,t}^k$	Binary variable to indicate if aircraft i is grounded and waiting for a type k check
$\theta_{i,t}^k$	Tolerance usage indicator of type k check of aircraft i on day t
Others:	
$\mathcal{S}^X(s_t, x_t)$	Transition function from s_t to s_{t+1} , $s_{t+1} = \mathcal{S}^X(s_t, x_t)$
$u_{i,t}^k$	Utilization of aircraft i on day t with respect to type k check
$\bar{u}_t^k$	Mean utilization of fleet on calendar day t for type k check
$V_t(s_t)$	The value of being in a state s_t

Step 1: Initialize Δu ($0 < \Delta u < 1$), $t \leftarrow t_0$, $S_t \leftarrow \{s_{t_0}\}$, $S_{t+1} \leftarrow \emptyset$

Step 2: Discretize the interval $[0, 1]$ with Δu : $0, \Delta u, 2\Delta u, \dots, 1$;

Step 3: For each *workable* $s_t \in S_t$:

Step 3.1: compute and sort the remaining utilization:

$$\tilde{R}_{1,t}^C, \dots, \tilde{R}_{N,t}^C \quad \tilde{R}_{i,t}^C \leq \tilde{R}_{i+1,t}^C \quad \tilde{R}_{i,t}^C \in \{R_{i,t}^C\}$$

$$\tilde{R}_{1,t}^A, \dots, \tilde{R}_{N,t}^A \quad \tilde{R}_{i,t}^A \leq \tilde{R}_{i+1,t}^A \quad \tilde{R}_{i,t}^A \in \{R_{i,t}^A\}$$

Step 3.2: For each action x_t of $s_t \in S_t$:

Step 3.2.1: Compute $C_t(s_t, x_t)$;

Step 3.2.2: Compute s_{t+1} using $s_{t+1} = \mathcal{S}^X(s_t, x_t)$;

Step 3.2.3: Check whether s_{t+1} is a *workable* state;

Step 3.2.4: Aggregate s_{t+1} according to

$$J_{t+1, \bar{u}_{t+1}^A, \bar{u}_{t+1}^C}^{\min}(s) = \min_{\{\bar{u}_{t+1}^A, \bar{u}_{t+1}^C\}} \left\{ J_{t+1, \bar{u}_{t+1}^A, \bar{u}_{t+1}^C}(s_{t+1}) \right\}$$

$$s_{t+1, \bar{u}_{t+1}^A, \bar{u}_{t+1}^C}^* = \operatorname{argmin}_{\{s, \bar{u}_{t+1}^A, \bar{u}_{t+1}^C\}} \left\{ J_{t+1, \bar{u}_{t+1}^A, \bar{u}_{t+1}^C}(s) \right\}$$

$$S_{t+1} = S_{t+1} \cup \left\{ s_{t+1, \bar{u}_{t+1}^A, \bar{u}_{t+1}^C}^* \right\}$$

Step 4: $t \leftarrow t + 1$;

Step 5: If $t \leq T$, go to **Step 3**;

$$\text{Else } s_{T+1, \bar{u}_{T+1}^A, \bar{u}_{T+1}^C}^* = \operatorname{argmin}_s \left\{ J_{T+1, \bar{u}_{T+1}^A, \bar{u}_{T+1}^C}(s) \right\};$$

$$x_T^*(s_T) = \operatorname{arg}_{x_T} \left\{ s_{T+1, \bar{u}_{T+1}^A, \bar{u}_{T+1}^C}^* = \mathcal{S}^X(s_T, x_T) \right\};$$

Step 6: Recover $x_{T-1}^*, x_{T-2}^*, \dots, x_{t_0+1}^*, x_{t_0}^*$

Algorithm 1: A dynamic programming based methodology for aircraft A- and C-check scheduling optimization.